

C# - Phần 9.6: Lambda Expression

Từ Delegate → Action / Func / Predicate → Lambda

Đây là phần rất quan trọng vì từ đây bạn sẽ bắt đầu thấy cú pháp C# hiện đại mà sau này dùng liên tục trong List<T>, LINQ, EF Core và ASP.NET Core.

Ý cốt lõi của Lambda: Lambda là cách viết ngắn gọn một method không cần đặt tên, thường được truyền trực tiếp vào Delegate.

1. Bắt đầu từ Predicate bạn vừa học

```
public static bool IsExpensive(Product product)
{
    return product.Price >= 1000;
}

products.Find(ProductCondition.IsExpensive);
```

Find() cần Predicate<Product>, tức là Product → bool.

Nếu method IsExpensive() chỉ dùng đúng ở chỗ này, tạo hẳn một method có tên có thể hơi dài dòng. Lambda cho phép viết trực tiếp:

```
Product? product =
    products.Find(
        product => product.Price >= 1000);
```

Phần product => product.Price >= 1000 chính là Lambda Expression.

2. Lambda này thực chất tương đương method nào?

```
product => product.Price >= 1000
```

Có thể hình dung tương đương:

```
public static bool IsExpensive(Product product)
{
    return product.Price >= 1000;
}
```

Lambda bỏ tên method, kiểu return viết rõ, từ khóa return và một số dấu ngoặc, nhưng ý nghĩa vẫn là: nhận Product → kiểm tra Price → trả bool.

3. Dấu => nghĩa là gì?

```
parameter => expression
```

Có thể đọc là: parameter → xử lý thành → expression.

```
number => number > 0
```

Nghĩa là nhận number và trả về kết quả number > 0.

4. Lambda với Predicate<T>

```
Predicate<int> check =  
    number => number > 0;
```

Predicate<int> yêu cầu nhận int và trả bool. Compiler nhìn kiểu bên trái và hiểu number là int.

```
Console.WriteLine(check(10));  
Console.WriteLine(check(-5));
```

Kết quả: True, False.

5. Vì sao không cần viết kiểu của number?

```
Predicate<int> check =  
    number => number > 0;
```

Compiler đã biết Predicate<int> nên Lambda phải nhận int. Bạn vẫn có thể viết rõ:

```
Predicate<int> check =  
    (int number) => number > 0;
```

Nhưng thông thường không cần.

6. Lambda với Action

Action<string> yêu cầu string → void.

```
public static void PrintMessage(string message)  
{  
    Console.WriteLine(message);  
}
```

Có thể dùng method có tên:

```
Action<string> print = PrintMessage;
```

Hoặc dùng Lambda:

```
Action<string> print =  
    message => Console.WriteLine(message);  
print("Hello");
```

7. Lambda với Func

Func<int, int> nghĩa là int → int.

```
public static int Square(int number)  
{  
    return number * number;  
}
```

Có thể thay bằng:

```
Func<int, int> square =  
    number => number * number;  
Console.WriteLine(square(5));
```

Kết quả: 25.

8. Lambda có nhiều parameter

```
Func<decimal, decimal, decimal> add =  
    (first, second) => first + second;
```

Khi có từ hai parameter trở lên, dùng dấu ngoặc:

```
(a, b) => a + b
```

9. Một parameter có thể bỏ ngoặc

```
number => number > 0
```

Hoặc:

```
(number) => number > 0
```

Với hai parameter thì phải có ngoặc: (a, b) => a + b.

10. Expression Lambda

```
number => number * number
```

Phía sau => chỉ có một biểu thức. Compiler hiểu giá trị biểu thức chính là giá trị return, nên không cần viết return.

```
Func<int, int> square =  
    number => number * number;
```

11. Statement Lambda

Nếu cần nhiều câu lệnh thì dùng { }.

```
Action<string> display = message =>  
{  
    Console.WriteLine("Message:");  
    Console.WriteLine(message);  
};
```

Với Func, nếu thân có nhiều statement thì phải dùng return:

```
Func<int, int, int> calculate =  
    (a, b) =>  
    {  
        int result = a + b;  
  
        Console.WriteLine($"Result: {result}");  
  
        return result;  
    };
```

12. Quay lại List<T>.Find()

Trước đây:

```
products.Find(  
    ProductCondition.IsExpensive);
```

Bây giờ:

```
Product? product =  
    products.Find(  
        product => product.Price >= 1000);
```

Find() vẫn cần Predicate<Product>. Chỉ thay cách cung cấp Predicate: từ method có tên sang Lambda.

13. So sánh ba cách

Cách 1 - Method riêng:

```
public static bool IsCheap(Product product)  
{  
    return product.Price < 100;  
}  
  
products.FindAll(IsCheap);
```

Cách 2 - Tạo biến Predicate:

```
Predicate<Product> condition =  
    product => product.Price < 100;  
  
products.FindAll(condition);
```

Cách 3 - Lambda trực tiếp:

```
products.FindAll(  
    product => product.Price < 100);
```

Cả ba cùng mô tả điều kiện Product → bool.

14. Ví dụ tìm Product theo Id

```
public Product? FindById(string id)  
{  
    return products.Find(  
        product => product.Id == id);  
}
```

Lambda product => product.Id == id:

- nhận Product
- lấy product.Id
- so sánh với id
- trả bool

Vì vậy nó phù hợp với Predicate<Product>.

15. Lambda với FindAll()

```
List<Product> cheapProducts =  
    products.FindAll(  
        product => product.Price < 100);
```

Đọc: lấy tất cả Product có Price nhỏ hơn 100.

16. Lambda với Exists()

```
bool hasExpensiveProduct =  
    products.Exists(  
        product => product.Price >= 1000);
```

Đọc: có ít nhất một Product có giá từ 1000 trở lên không?

17. Lambda với RemoveAll()

```
int removedCount =  
    products.RemoveAll(  
        product => product.Price < 100);
```

Đọc: xóa tất cả Product có giá nhỏ hơn 100.

18. Lambda thường cần một kiểu đích

Ví dụ biểu thức:

```
number => number > 0
```

thường cần compiler biết nó đang được dùng như Delegate nào.

```
Predicate<int> check =  
    number => number > 0;  
  
Func<int, bool> check =  
    number => number > 0;  
  
numbers.Find(  
    number => number > 0);
```

Trong Find(), compiler biết parameter cần Predicate<int> nên hiểu Lambda theo kiểu đó.

19. Lambda không phải Delegate

Lambda là cú pháp biểu diễn một hàm ẩn danh. Nó có thể được chuyển sang các Delegate có chữ ký phù hợp.

```
Predicate<int> p =  
    x => x > 0;  
  
Func<int, bool> f =  
    x => x > 0;
```

Cùng một Lambda về hình thức nhưng có thể được gán cho các Delegate khác nhau nếu chữ ký tương thích.

20. Liên hệ toàn bộ Phần 9 đến hiện tại

```
public delegate bool CheckNumber(int number);
```

→ custom Delegate

```
Predicate<int>
```

→ Delegate có sẵn

```
number => number > 0
```

→ Lambda Expression

Có thể hình dung: Custom Delegate → Action / Func / Predicate → Lambda Expression.

Lambda không loại bỏ Delegate. Nó chỉ giúp viết hành vi ngắn hơn.

21. Tại sao Lambda cực kỳ quan trọng?

Vì sau này khi học LINQ, bạn sẽ gặp rất nhiều:

```
products.Where(  
    product => product.Price >= 1000);  
  
products.Select(  
    product => product.Name);  
  
products.OrderBy(  
    product => product.Price);
```

Hiện tại chưa cần học LINQ. Chỉ cần hiểu `product => ...` là một hành vi/hàm không tên được truyền vào API.

Tóm tắt 9.6

- Lambda là cách viết hàm ẩn danh ngắn gọn.
- Cú pháp cơ bản: `parameter => expression`.
- Một parameter có thể bỏ ngoặc; nhiều parameter cần ngoặc.
- Expression Lambda thường không cần `return`.
- Statement Lambda dùng `{ }` và với Func thường cần `return`.
- Lambda thường được chuyển thành Action, Func, Predicate hoặc Delegate phù hợp.

Bài tập 9.6 - Lambda Expression

Có thể tiếp tục dùng Product của bài Predicate:

```
public class Product  
{  
    public string Name { get; }  
    public decimal Price { get; }  
  
    public Product(  
        string name,  
        decimal price)  
    {  
        Name = name;  
        Price = price;  
    }  
  
    public override string ToString()  
    {  
        return $"{Name} - {Price}";  
    }  
}
```

Trong Program, tạo:

```
List<Product> products = new List<Product>
{
    new Product("Mouse", 20),
    new Product("Keyboard", 80),
    new Product("Monitor", 300),
    new Product("Laptop", 1500)
};
```

Thực hiện lần lượt:

- 1. Predicate<int> dùng Lambda kiểm tra số > 0.
- 2. Func<int, int> dùng Lambda tính bình phương.
- 3. Func<int, int, int> dùng Lambda cộng hai số.
- 4. Action<string> dùng Lambda in message ra Console.
- 5. Dùng Lambda với products.Find(...) để tìm Product có Price >= 1000.
- 6. Dùng Lambda với products.FindAll(...) để tìm Product có Price < 100.
- 7. Dùng Lambda với products.Exists(...) để kiểm tra có Product Price >= 1000 hay không.
- 8. Viết một Statement Lambda có ít nhất hai câu lệnh trong thân { }.

Khung để tự làm:

```
Predicate<int> isPositive =
    /* Lambda */;

Func<int, int> square =
    /* Lambda */;

Func<int, int, int> add =
    /* Lambda */;

Action<string> display =
    /* Lambda */;

Product? expensiveProduct =
    products.Find(
        /* Lambda */);
```

Câu hỏi cuối bài

- 1. Lambda Expression là gì?
- 2. $x \Rightarrow x > 0$ nhận kiểu gì và trả về kiểu gì nếu được gán cho Predicate<int>?
- 3. Vì sao products.Find(product => product.Price >= 1000) không cần tạo method IsExpensive() riêng?
- 4. Expression Lambda và Statement Lambda khác nhau thế nào?
- 5. Lambda có thay thế hoàn toàn Delegate không? Vì sao?

Mục tiêu của bài: khi thấy $x \Rightarrow \dots$ hãy phản xạ rằng "đây là một hành vi/method không tên được truyền cho Delegate hoặc API đang cần Delegate".